

cs282 specialContributionE-Mamba

👤 Suppose I'm an undergrad with some knowledge of machine learning (have taken classes relevant to linear algebra, multivariable calculus, probability, and intro to ML), and suppose you are a tutor who is proficient in deep learning. Can you teach me the entire concept of Mamba (state space model) step by step, ideally with some previous relevant model architecture like RNN that sort of inspired the development of Mamba. And feel free to add some quiz throughout the process so that you can make sure I fully understand the concepts and/or adjust your teaching materials according to my performance on the quiz.

Thought: Architected comprehensive tutorial scaffolding RNNs through Mamba with embedded assessments.

Great! I'm excited to teach you about Mamba. This will be a journey from classical RNNs to modern state space models. Let's build this step by step.

Part 1: The Foundation - RNNs and Their Limitations

1.1 Quick RNN Recap

You probably know RNNs, but let's establish our foundation. An RNN processes sequences by maintaining a hidden state h_t :

$$\begin{aligned}h_t &= f(W_h * h_{t-1} + W_x * x_t + b) \\ y_t &= W_y * h_t\end{aligned}$$

Able to identify RNN as its precursor and also justify the limitations of RNN, providing a good motivation of developing SSM

The key idea: **hidden state carries information across time steps.**

1.2 The Core Problems with RNNs

RNNs have three major issues that motivate everything that follows:

1. **Sequential Processing:** You MUST compute h_1 , then h_2 , then h_3 ...
Cannot parallelize during training.
2. **Vanishing/Exploding Gradients:** As sequences get long, gradients either vanish ($\rightarrow 0$) or explode ($\rightarrow \infty$) due to repeated multiplication through time.
3. **Limited Long-Range Dependencies:** Hard to remember information from 1000+ steps ago.

LSTMs and GRUs helped with problem #2 and #3, but **problem #1 remained:** they're still inherently sequential.

1.3 Enter Transformers

Transformers solved parallelization with self-attention - you can compute all positions simultaneously! But they introduced a NEW problem:

Quadratic Complexity: $O(L^2)$ time and memory for sequence length L , because every token attends to every other token.

This makes Transformers struggle with very long sequences (100K+ tokens).

[Also point out the subsequent development and limitation of Transformer](#)

Quick Check Quiz #1

Before we move on, let me check your understanding:

Q1: Why can't we parallelize RNN training across the sequence length dimension?
(Be specific about the dependency)

Q2: Transformers can be parallelized during training. What's their main limitation for very long sequences?

Q3: If you have a sequence of length 10,000, approximately how many attention computations does a single Transformer layer perform? (Order of magnitude is fine)

Please answer these, and I'll adjust my teaching based on your responses! Then we'll move to the exciting part: how state space models try to get the best of both worlds.

- ⊙ A1: Because each hidden state at a specific time step will depend on the hidden state at its previous time step. A2: Quadratic complexity of the attention mechanism A3: 10^8

Thought: Validated student comprehension and escalated pedagogical depth.

Perfect! All three answers are correct. You've got a solid grasp of the fundamentals. Let's dive deeper.

Part 2: State Space Models (SSMs) - The Core Idea

2.1 The Key Insight

State space models ask: "What if we could model sequences like RNNs (with a hidden state) BUT make them parallelizable like Transformers?"

The answer comes from **control theory** and continuous-time systems.

The insight is highly intuitive!

2.2 Continuous-Time State Space Model

In classical control theory, a state space model is defined by these equations:

$$\begin{aligned} \mathbf{h}'(t) &= \mathbf{A} \cdot \mathbf{h}(t) + \mathbf{B} \cdot \mathbf{x}(t) && [\text{State equation}] \\ \mathbf{y}(t) &= \mathbf{C} \cdot \mathbf{h}(t) + \mathbf{D} \cdot \mathbf{x}(t) && [\text{Output equation}] \end{aligned}$$

Where:

- **$\mathbf{x}(t)$** : Input signal (continuous time)

- **$h(t)$** : Hidden state (N-dimensional)
- **$h'(t)$** : Derivative of hidden state
- **$y(t)$** : Output signal
- **A** ($N \times N$): State transition matrix
- **B** ($N \times 1$): Input projection matrix
- **C** ($1 \times N$): Output projection matrix
- **D** (scalar): Skip connection (often 0)

Key intuition: The hidden state **$h(t)$** evolves continuously according to a differential equation controlled by matrix **A** .

2.3 Why This Helps: Discretization

Here's the magic - we can **discretize** this continuous system to get a recurrent form:

$$\begin{aligned} h_t &= \bar{A} \cdot h_{t-1} + \bar{B} \cdot x_t \\ y_t &= C \cdot h_t \end{aligned}$$

Able to recover the insight about discretization and thus being able to use convolution to parallel the process!

Where $\bar{A}$ and $\bar{B}$ are discrete versions of A and B (using a step size Δ):

- $\bar{A} = \exp(\Delta A)$ [matrix exponential]
- $\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$

This looks like an RNN! But here's the trick...

2.4 The Convolutional Trick

We can also express the same system as a **convolution**:

$$y = x \otimes K$$



Where K is a filter derived from (A, B, C) : $\mathbf{K} = (\mathbf{C} \cdot \bar{\mathbf{B}}, \mathbf{C} \cdot \bar{\mathbf{A}} \cdot \bar{\mathbf{B}}, \mathbf{C} \cdot \bar{\mathbf{A}}^2 \cdot \bar{\mathbf{B}}, \mathbf{C} \cdot \bar{\mathbf{A}}^3 \cdot \bar{\mathbf{B}}, \dots)$

Why does this matter? **Convolutions can be parallelized using FFT!**

So we get:

- **Training:** Use convolution form $\rightarrow$ fully parallel, $O(L \log L)$ with FFT
- **Inference:** Use recurrent form $\rightarrow$ efficient autoregressive generation, $O(1)$ per step

Quick Check Quiz #2

Q1: In the continuous state equation $h'(t) = A \cdot h(t) + B \cdot x(t)$, what does matrix A control intuitively?

Q2: Why can we use convolution during training but recurrence during inference, yet get the same output?

Q3: If convolution is $O(L \log L)$ and Transformer attention is $O(L^2)$, at roughly what sequence length does the SSM become more efficient? (Rough estimate)

Take your time with these - they're getting conceptually deeper!

These are all insightful questions! But I wanted more details about the derivation of the discretization and also the convolution part so I asked Claude to elaborate on it more later.

👤 Can you elaborate more on this, with more detail explanations and equation derivation

Thought: Orchestrated comprehensive mathematical treatment with rigorous derivations.

Excellent! Let's dig deep into the mathematics. I'll go slow and derive everything carefully.

Part 2 (Detailed): State Space Models - Mathematical Deep Dive

2.1 Continuous-Time State Space - Intuition and Meaning

Let's start with the continuous equations:

$$\begin{aligned} \mathbf{h}'(t) &= \mathbf{A} \cdot \mathbf{h}(t) + \mathbf{B} \cdot \mathbf{x}(t) && [\text{State equation}] \\ y(t) &= \mathbf{C} \cdot \mathbf{h}(t) + \mathbf{D} \cdot \mathbf{x}(t) && [\text{Output equation}] \end{aligned}$$

What does each term mean?

$\mathbf{h}'(t) = d\mathbf{h}/dt$: The rate of change of the hidden state. Think of $\mathbf{h}(t)$ as a particle's position, then $\mathbf{h}'(t)$ is its velocity.

$\mathbf{A} \cdot \mathbf{h}(t)$: This term determines how the state evolves on its own (without input).

- If $\mathbf{A}$ is negative, the state decays over time (think: friction, forgetting)

- The eigenvalues of A determine stability and how fast different dimensions evolve
- **This is the "memory" mechanism** - A controls what gets remembered and forgotten

$B \cdot x(t)$: How the current input affects the state change

- B projects the input into the state space
- Determines what information from the input is stored

$C \cdot h(t)$: Projects the hidden state to output

- Determines what aspects of the state we actually observe

$D \cdot x(t)$: Direct skip connection from input to output (often set to 0 in SSMs)

Example: Simple 1D Case

Let's say $N=1$ (1-dimensional state), $A=-1$, $B=1$, $C=1$, $D=0$:

$$\begin{aligned} h'(t) &= -h(t) + x(t) \\ y(t) &= h(t) \end{aligned}$$

This is a **leaky integrator**: the state accumulates the input but also decays exponentially. If $x(t)=0$, then $h(t) = h(0) \cdot e^{(-t)}$, decaying to zero.

2.2 Discretization - Detailed Derivation

We need to convert the continuous system to discrete time steps. Let Δ be our step size.

Step 1: Solve the Differential Equation

Starting from: **$h'(t) = A \cdot h(t) + B \cdot x(t)$**

This is a first-order linear ODE. Assuming $x(t)$ is constant over $[k\Delta, (k+1)\Delta]$, the solution is:

$$\mathbf{h}(t) = \mathbf{e}^{\{\mathbf{A}(t-t_0)\}} \cdot \mathbf{h}(t_0) + \int_{t_0}^t \mathbf{e}^{\{\mathbf{A}(t-\tau)\}} \cdot \mathbf{B} \cdot \mathbf{x}(\tau) d\tau$$

This comes from the **variation of parameters** method. The first term is the homogeneous solution, the second is the particular solution.

Step 2: Evaluate at Discrete Time Points

Let $t_0 = k\Delta$ and $t = (k+1)\Delta$. Assuming x is constant ($= x_k$) over this interval:

$$\mathbf{h}_{k+1} = \mathbf{e}^{\{\mathbf{A}\Delta\}} \cdot \mathbf{h}_k + \left(\int_0^\Delta \mathbf{e}^{\{\mathbf{A}s\}} ds \right) \cdot \mathbf{B} \cdot \mathbf{x}_k$$

Now we need to evaluate that integral:

$$\int_0^\Delta \mathbf{e}^{\{\mathbf{A}s\}} ds$$

For a matrix exponential, if A is invertible:

$$\int_0^\Delta \mathbf{e}^{\{\mathbf{A}s\}} ds = \mathbf{A}^{-1} (\mathbf{e}^{\{\mathbf{A}\Delta\}} - \mathbf{I})$$

where $\mathbf{I}$ is the identity matrix.

Step 3: Define Discrete Parameters

$$\bar{\mathbf{A}} = \mathbf{e}^{\{\mathbf{A}\Delta\}} \text{ (matrix exponential)}$$

$$\bar{\mathbf{B}} = \mathbf{A}^{-1} (\mathbf{e}^{\{\mathbf{A}\Delta\}} - \mathbf{I}) \cdot \mathbf{B} = \mathbf{A}^{-1} (\bar{\mathbf{A}} - \mathbf{I}) \cdot \mathbf{B}$$

For small Δ , we can approximate using Taylor series:

- $\mathbf{e}^{\{\mathbf{A}\Delta\}} \approx \mathbf{I} + \mathbf{A}\Delta + (\mathbf{A}\Delta)^2/2! + \dots$
- So $\bar{\mathbf{A}} \approx \mathbf{I} + \mathbf{A}\Delta$ (first order approximation)
- And $\bar{\mathbf{B}} \approx \Delta \mathbf{B}$

Final Discrete Recurrence:

$$h_k = \bar{A} \cdot h_{k-1} + \bar{B} \cdot x_k$$

$$y_k = C \cdot h_k (+ D \cdot x_k)$$

This is our recurrent form - looks exactly like an RNN!

2.3 Convolution Form - Detailed Derivation

Now for the magic trick. Let's unroll the recurrence starting from $h_0 = 0$:

Step k=1: $h_1 = \bar{A} \cdot h_0 + \bar{B} \cdot x_1 = \bar{B} \cdot x_1$ $y_1 = C \cdot h_1 = \mathbf{C \cdot \bar{B} \cdot x_1}$

Step k=2: $h_2 = \bar{A} \cdot h_1 + \bar{B} \cdot x_2 = \bar{A} \cdot \bar{B} \cdot x_1 + \bar{B} \cdot x_2$ $y_2 = C \cdot h_2 = \mathbf{C \cdot \bar{A} \cdot \bar{B} \cdot x_1 + C \cdot \bar{B} \cdot x_2}$

Step k=3: $h_3 = \bar{A} \cdot h_2 + \bar{B} \cdot x_3 = \bar{A}^2 \cdot \bar{B} \cdot x_1 + \bar{A} \cdot \bar{B} \cdot x_2 + \bar{B} \cdot x_3$ $y_3 = C \cdot h_3 = \mathbf{C \cdot \bar{A}^2 \cdot \bar{B} \cdot x_1 + C \cdot \bar{A} \cdot \bar{B} \cdot x_2 + C \cdot \bar{B} \cdot x_3}$

Pattern Recognition

Do you see the pattern? Each output y_k is a weighted sum of all previous inputs:

$$y_k = \sum_{i=1}^k C \cdot \bar{A}^{k-i} \cdot \bar{B} \cdot x_i$$

Let's define the **SSM convolution kernel**:

$\bar{K} = (\bar{K}_0, \bar{K}_1, \bar{K}_2, \bar{K}_3, \dots)$ where:

- $\bar{K}_0 = C \cdot \bar{B}$
- $\bar{K}_1 = C \cdot \bar{A} \cdot \bar{B}$
- $\bar{K}_2 = C \cdot \bar{A}^2 \cdot \bar{B}$
- $\bar{K}_3 = C \cdot \bar{A}^3 \cdot \bar{B}$
- $\bar{K}_k = C \cdot \bar{A}^k \cdot \bar{B}$

Then we can write:

$$y = x \otimes \bar{K}$$

Where $\otimes$ denotes discrete convolution:

$$y_k = \sum_{i=0}^{k-1} \bar{K}_i x_{k-i}$$

Why Convolution is Efficient

Convolution in time domain is $O(L^2)$, BUT:

Convolution Theorem: Convolution in time = pointwise multiplication in frequency domain

$$y = x \otimes K \iff Y = X \odot \hat{K}$$

Where $Y, X, \hat{K}$ are Fourier transforms.

Algorithm:

1. Compute FFT of x : $O(L \log L)$
2. Compute FFT of $\bar{K}$: $O(L \log L)$ - can precompute!
3. Pointwise multiply: $O(L)$
4. Inverse FFT: $O(L \log L)$

Total: $O(L \log L)$ instead of $O(L^2)$!

2.4 Why Are Recurrent and Convolutional Forms Equivalent?

This is subtle but important. They compute **the same function**, but:

Recurrent Form (Inference):

$$h_k = \bar{A} \cdot h_{k-1} + \bar{B} \cdot x_k$$

$$y_k = C \cdot h_k$$

- Requires previous state h_{k-1}
- **Sequential:** must compute $k=1$, then $k=2$, etc.
- **Good for generation:** $O(1)$ per step once you have h_{k-1}
- **Constant memory:** just store h (size N)

Convolutional Form (Training):

$$y = x \otimes \bar{K} \text{ where } \bar{K}_k = C \cdot \bar{A}^k \cdot \bar{B}$$

- No sequential dependency!
- **Parallel:** can compute all y_k simultaneously using FFT
- **Good for training:** process entire sequence at once
- **More memory:** need to store $\bar{K}$ (length L) and intermediate FFT results

Key Insight

Both forms implement the same linear time-invariant (LTI) system. The recurrence defines the system, and the convolution kernel $\bar{K}$ is just the **impulse response** of that system.

In signal processing terms: if you input a unit impulse δ at $t=0$, the output sequence is exactly $\bar{K}$!



Deeper Understanding Quiz #3



Q1: In the discretization, why do we need the matrix exponential $e^{\{A\Delta\}}$ rather than just using $I + A\Delta$?

Q2: The convolution kernel is $\bar{K}_k = C \cdot \bar{A}^k \cdot \bar{B}$. If the largest eigenvalue of $\bar{A}$ has magnitude > 1 , what happens to $\bar{K}_k$ as $k \rightarrow \infty$? What about if all eigenvalues have magnitude < 1 ?

Q3: During training, we use convolution (parallel). During inference, we use recurrence (sequential). Why don't we use convolution for inference too?

Take your time - these questions probe deep understanding of the mathematics!

⊙ A1: You can only use $I + A\Delta$ to approximate $e^{A\Delta}$ for very small Δ . A2: for > 1 , $\tilde{K}_k$ will explode as $k \rightarrow \infty$, while for < 1 , $\tilde{K}_k$ will vanish as $k \rightarrow \infty$. A3: cause during inference we receive tokens (or timesteps) one by one and need to update the hidden state incrementally, and we only need the previous state and current input to generate the next step (so just need to store the previous state in memory), which is highly computationally efficient with $O(1)$ complexity as opposed to $O(L \log L)$ complexity using convolution.

Thought: Validated student mastery, charted progression toward HiPPO framework.

Excellent! All three answers are spot-on. You clearly grasp the mathematical foundations. Your answer to Q3 especially shows you understand the practical computational tradeoffs.

Your insight about A2 is particularly important - it leads us to a critical question:

How do we choose A to make the system stable AND able to capture long-range dependencies?

This is where the breakthrough happens.

Part 3: The HiPPO Framework - Making SSMs Remember

3.1 The Challenge: Long-Range Dependencies

Remember from Q2: if eigenvalues of $\tilde{A}$ have magnitude < 1 , then $\tilde{K}_k \rightarrow 0$ as $k \rightarrow \infty$.

This means **the system forgets!** Inputs from far in the past have vanishing influence.

We only very briefly mention HiPPO during lecture, and here it provided a much more detailed explanation which I think is also insightful

But we WANT to remember long contexts. So we face a dilemma:

- **Stable system** ($|\text{eigenvalues}| < 1$) $\rightarrow$ forgets the past
- **Unstable system** ($|\text{eigenvalues}| \geq 1$) $\rightarrow$ exploding gradients

HiPPO (High-order Polynomial Projection Operators) solves this elegantly.

3.2 The HiPPO Idea: Polynomial Approximation

Instead of arbitrary A , HiPPO derives A from a principled mathematical framework:

Goal: At each time t , maintain state $h(t)$ that can **reconstruct** the input history $x(s)$ for $s \leq t$ using polynomials.

Specifically, project the history onto a basis of orthogonal polynomials (like Legendre polynomials).

Mathematical Setup

Define a measure $\mu(t)$ over history (e.g., uniform, or exponentially decaying). At time t , we want to find coefficients $c(t)$ that minimize:

$$\min \int_0^t |x(s) - \sum_i c_i(t) \cdot p_i(s)|^2 \mu(t-s) ds$$

where p_i are orthogonal polynomials (degree i).

The Result: HiPPO Matrix

Through calculus of variations, this optimization problem gives us a differential equation for how coefficients evolve:

$$\mathbf{c}'(t) = \mathbf{A} \cdot \mathbf{c}(t) + \mathbf{B} \cdot \mathbf{x}(t)$$

This is exactly our state space form! And the specific structure of A depends on the measure μ .

3.3 HiPPO-LegS: The Most Common Choice

For **uniform measure** over a sliding window $[t-\theta, t]$, the HiPPO matrix is:

$$\mathbf{A}_{\{nk\}} = -$$

This part goes beyond the lecture materials but seems correct

$$\begin{cases} (2n+1)^{(1/2)} (2k+1)^{(1/2)} & \text{if } n > k \\ n+1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases}$$

And $\mathbf{B}_n = (2n+1)^{(1/2)}$

This looks complicated, but key properties:

1. **A is lower triangular** with negative diagonal
2. **All eigenvalues are negative real numbers**
3. When discretized, $\bar{A}$ has eigenvalues inside unit circle (stable!)
4. **But:** The system can still remember long dependencies because of the special structure

Why It Works: The Geometry

The HiPPO initialization creates eigenvalues that are:

- All **negative** (stable in continuous time)
- But **spread out** along negative real axis
- Some eigenvalues close to 0 (slow decay = long memory)
- Some eigenvalues very negative (fast decay = capture high-frequency details)

After discretization with $\bar{A} = \exp(\Delta A)$:

- Eigenvalues map to inside unit circle
- But distributed in a way that balances memory and stability

3.4 Intuition: Why Does This Remember Better?

Think of the state h as storing **compressed information** about the input history using orthogonal polynomials.

- Low-degree polynomial coefficients → capture smooth, long-term trends
- High-degree polynomial coefficients → capture recent, detailed variations

By maintaining ALL coefficients, the system balances:

- **Long-term memory** (low-degree terms decay slowly)
- **Recent detail** (high-degree terms are frequently updated)

This is fundamentally different from an RNN's h , which is just "whatever the network learned to store" without this principled structure.

3.5 The S4 Model (Structured State Space)

HiPPO gave us good A , but there's still a problem: computing $\tilde{A} = \exp(\Delta A)$ and the convolution kernel $\tilde{K}$ is expensive!

S4 (Structured State Spaces) introduces computational tricks:

Key Innovation 1: Diagonalization

Instead of working with A directly, diagonalize it:

$$A = V \cdot \Lambda \cdot V^{-1}$$

where Λ is diagonal (eigenvalues).

$$\text{Then: } \tilde{A} = \exp(\Delta A) = V \cdot \exp(\Delta \Lambda) \cdot V^{-1}$$

And $\exp(\Delta \Lambda)$ is easy - just exponentiate each eigenvalue!

Key Innovation 2: Cauchy Kernel Trick

The convolution kernel becomes:

$$\bar{\mathbf{K}}_k = \mathbf{C} \cdot \bar{\mathbf{A}}^k \cdot \bar{\mathbf{B}} = \mathbf{C} \cdot \mathbf{V} \cdot \mathbf{\Lambda}^k \cdot \mathbf{V}^{-1} \cdot \bar{\mathbf{B}}$$

For computing $\bar{\mathbf{K}}$ efficiently, S4 uses the fact that this can be computed as a **Cauchy matrix** product, which has fast $O(N \log N)$ algorithms.

Result

S4 can:

- Handle very large state dimensions ($N = 64, 128$, even 256)
- Process very long sequences ($L = 16K+$)
- Train efficiently with HiPPO initialization
- Achieve strong performance on long-range dependency tasks

Conceptual Quiz #4

Q1: HiPPO initializes $\mathbf{A}$ to maintain polynomial coefficients of the input history. Why is this better than random initialization for long-range dependencies?

Q2: After discretization, HiPPO matrices produce $\bar{\mathbf{A}}$ with eigenvalues inside the unit circle (stable), yet can remember long contexts. How is this different from a simple RNN with forget gate near 1?

Q3: S4 uses diagonalization $\mathbf{A} = \mathbf{V} \cdot \mathbf{\Lambda} \cdot \mathbf{V}^{-1}$ for efficiency. What's the computational advantage when computing $\bar{\mathbf{A}}^k = (\mathbf{V} \cdot \mathbf{\Lambda} \cdot \mathbf{V}^{-1})^k$?

These questions test whether you understand WHY these design choices matter, not just WHAT they are.



⊙ A1: A randomly initialized A matrix defines a random, unstable dynamical system. When you feed it a sequence, the hidden state h will either decay to zero or explode to infinity as range becomes longer (In either case, all information about the distant past is completely lost). However, HiPPO initialized structured A such that h is now a highly compressed, useful summary of the entire past. The model doesn't just "remember" the last few tokens; it maintains a mathematical approximation of everything it has seen, allowing it to capture dependencies across even longer time steps. A2: HiPPO's structured A yields diverse, orthogonal modes of memory, while an RNN's near-identity dynamics yield redundant, single-timescale memory. A3: $\exp(\Delta \Lambda)$ can now be easily computed via exponentiate each eigenvalue (scaler)

Thought: Lauded exceptional understanding and prepared transition to advanced concepts.

Outstanding! Your answers demonstrate exceptionally deep understanding. Let me highlight what's impressive:

A1: You nailed the key insight - HiPPO creates a *compressed, useful summary* using orthogonal polynomial basis. This is the fundamental difference from random initialization. You're thinking like a mathematician here.

A2: Brilliant! "Diverse, orthogonal modes vs. redundant, single-timescale memory" - this captures the essence perfectly. An RNN with forget gate ≈ 1 creates one exponential decay rate, while HiPPO creates a *spectrum* of decay rates through its eigenvalue distribution.

A3: Exactly right. I'd add one detail: Λ^k is trivial to compute because (diagonal matrix)^k = diagonal matrix with each entry raised to k-th power. No matrix multiplications needed!

You're ready for Mamba. But first, we need to understand **the key limitation that S4 still has**.

Part 4: The Path to Mamba - S4's Limitation

4.1 S4's Critical Weakness: Time-Invariance

Remember our discrete SSM:

$$\begin{aligned}h_k &= \bar{A} \cdot h_{k-1} + \bar{B} \cdot x_k \\y_k &= C \cdot h_k\end{aligned}$$

Successfully identify the key weakness of the S4 structure!

Notice: $\bar{A}$, $\bar{B}$, C are **fixed parameters**, independent of the input!

This means:

- The same matrices process token 1, token 100, and token 10,000
- The dynamics are **time-invariant** and **input-invariant**
- The "forgetting rate" and "what to remember" are hardcoded

Why This Is Limiting

Consider reading text:

- Some tokens are important (names, key facts) → should remember strongly
- Some tokens are filler (common words like "the", "and") → can forget quickly
- Context matters: "bank" (river vs. money) needs different processing

S4 cannot adapt its dynamics based on content! It treats all inputs with the same filtering/memory mechanism.

This is like having a fixed attention pattern in Transformers - you lose the content-based routing that makes attention powerful.



4.2 The Mamba Innovation: Selective State Spaces

Mamba's key idea: Make the SSM parameters **input-dependent** (selective).

Instead of fixed $\bar{A}$, $\bar{B}$, C , we make them functions of the input:

```
 $\bar{B}_k = \text{Linear\_B}(x_k)$       [input-dependent!]  
 $C_k = \text{Linear\_C}(x_k)$       [input-dependent!]  
 $\Delta_k = \text{Broadcast}(\text{Linear\_}\Delta(x_k))$  [input-dependent step size!]
```

And then compute $\bar{A}$ from Δ :

```
 $\bar{A}_k = \exp(\Delta_k \cdot A)$ 
```

Where A is still the fixed HiPPO matrix, but Δ **varies per input!**

What Does This Enable?

Δ_k (step size) controls memory:

- Large $\Delta \rightarrow$ larger steps $\rightarrow$ faster forgetting
- Small $\Delta \rightarrow$ smaller steps $\rightarrow$ slower forgetting (more memory)

B_k controls what information enters state:

- Important inputs $\rightarrow$ large $B \rightarrow$ strong encoding
- Unimportant inputs $\rightarrow$ small $B \rightarrow$ weak encoding

C_k controls what information is read out:

- Relevant context $\rightarrow$ large $C \rightarrow$ strong output
- Irrelevant context $\rightarrow$ small $C \rightarrow$ suppressed output

The model learns to be **selective** about what to remember and when!

4.3 Mamba Architecture Details

Let me show you the full Mamba block:

Input Processing

$x \in \mathbb{R}^{(L \times D)}$ [sequence length L , model dimension D]

Step 1: Expand and Split (like Gated MLPs)

$x' = \text{Linear}(x) \in \mathbb{R}^{(L \times 2D)}$
 $x_1, x_2 = \text{split}(x')$ [each $\in \mathbb{R}^{(L \times D)}$]

Step 2: Compute SSM Parameters (The Selective Part!)

$B = \text{Linear}_B(x_1) \in \mathbb{R}^{(L \times N)}$ [N = state dimension]
 $C = \text{Linear}_C(x_1) \in \mathbb{R}^{(L \times N)}$
 $\Delta = \text{Broadcast}(\text{Softplus}(\text{Linear}_\Delta(x_1))) \in \mathbb{R}^{(L \times D)}$

Key: Each position gets its own B, C, Δ !

Step 3: Discretize Per-Input

For each position k :

$\bar{A}_k = \exp(\Delta_k \cdot A)$
 $\bar{B}_k = (\Delta_k \cdot A)^{(-1)} \cdot (\bar{A}_k - I) \cdot (\Delta_k \cdot B_k)$

Step 4: Selective Scan (The Core Operation)

$$h_k = \bar{A}_k \cdot h_{k-1} + \bar{B}_k \cdot x_{1,k}$$

$$y_k = C_k \cdot h_k$$

Notice: $\bar{A}$ and $\bar{B}$ vary with k now!

Step 5: Gating and Output

$$y = y \odot \text{SiLU}(x_2) \quad [\text{gating mechanism}]$$

$$\text{output} = \text{Linear}(y)$$

Correctly identify that FFT convolution can no longer be used (as mentioned during lecture), also further explain Mamba's solution to that

4.4 The Computational Challenge

Here's the problem: **we can't use FFT convolution anymore!**

Why? The convolution kernel $\bar{K}_k = C_k \cdot \bar{A}_k^k \cdot \bar{B}_k$ is now **different for each input sequence**. We can't precompute it!

We're forced back to recurrent processing... or are we?

Mamba's Solution: Hardware-Aware Parallel Scan

Mamba uses a clever algorithm called **parallel scan** (also known as **associative scan**):

The recurrence $h_k = \bar{A}_k \cdot h_{k-1} + \bar{B}_k \cdot x_k$ can be parallelized if we view it as:

$$h_k = \bar{A}_k \cdot (\dots (\bar{A}_2 \cdot (\bar{A}_1 \cdot h_0 + \bar{B}_1 \cdot x_1) + \bar{B}_2 \cdot x_2) \dots) + \bar{B}_k \cdot x_k$$

This has **associative structure**, allowing $O(\log L)$ depth parallel computation (though with more operations).

Combined with **kernel fusion** (computing everything in a single GPU kernel without materializing intermediate results in HBM), Mamba achieves:

- **Training:** $\sim O(L)$ time, competitive with Transformers
- **Inference:** $O(1)$ per step like RNNs (use recurrent form)

4.5 Mamba vs. Alternatives - Summary Table

Model	Complexity (Training)	Complexity (Inference)	Long Context	Content-Based
RNN/LSTM	$O(L)$ sequential	$O(1)$ per step	✗ (vanishing)	✓ (gating)
Transformer	$O(L^2)$ parallel	$O(L)$ per step	✓	✓ (attention)
S4	$O(L \log L)$ parallel	$O(1)$ per step	✓ (HiPPO)	✗ (fixed)
Mamba	$O(L)$ parallel	$O(1)$ per step	✓ (HiPPO)	✓ (selective)

Mamba gets the best of all worlds!

Great summary table, although didn't reflect that S4 & Mamba has better long context ability compared to Transformer

Synthesis Quiz #5

Q1: In S4, the SSM parameters are fixed, so we can compute $\tilde{K} = (C \cdot \bar{B}, C \cdot \bar{A} \cdot \bar{B}, C \cdot \bar{A}^2 \cdot \bar{B}, \dots)$ once and use FFT. Why can't we do this in Mamba?

Q2: The step size Δ_k varies per token in Mamba. If the model learns to produce large Δ for certain tokens, what does this mean about how those tokens are processed?

Q3: Mamba uses both selective SSM (input-dependent) and gating ($y \odot \text{SiLU}(x_2)$). How do these two mechanisms differ in what they control?

Take your time - these probe the deepest layer of understanding!

👤 A1: because for mamba each position has different A,B,and C. A2: those tokens will be forgotten faster. A3: Selective SSM controls the internal memory of the SSM (what to remember and what to forget), Gating controls the output of the SSM (how much of that memory to actually use and pass on).

Thought: Validated sophisticated understanding and prepared advanced synthesis.

Perfect! All three answers are excellent. Your answer to A3 is particularly sophisticated - you've identified the two-level control mechanism:

1. **Selective SSM** (internal): Controls the state space dynamics - what information flows into h , how it decays, what gets read out
2. **Gating** (external): Controls the final output - acts as a multiplicative filter on the SSM output

This dual mechanism gives Mamba fine-grained control over information flow.

You've mastered the conceptual foundations! Now let's complete the picture with architectural details and practical considerations.

Part 5: Mamba Architecture - Complete Picture

5.1 Full Mamba Block (Detailed)

Let me show you the exact architecture with dimensions:

Seems a lot of followed contents are a bit beyond scope of what's been covered in lectures. But I think they helped me gain a deeper understanding of the Mamba design, and I didn't identify obvious fact errors.

```

class MambaBlock:
    def __init__(self, d_model, d_state, d_conv):
        # d_model: model dimension (e.g., 768)
        # d_state: SSM state dimension N (e.g., 16)
        # d_conv: local convolution width (e.g., 4)

        self.in_proj = Linear(d_model, 2*d_model) # expand

        # 1D convolution for local context
        self.conv1d = Conv1d(d_model, d_model,
kernel_size=d_conv, groups=d_model)

        # SSM parameters
        self.x_proj = Linear(d_model, d_state*2 + d_conv) # B,
C, Δ

        self.dt_proj = Linear(d_conv, d_model)

        # Fixed HiPPO A matrix
        self.A_log = Parameter(...) # log of A for numerical
stability
        self.D = Parameter(...) # skip connection

        self.out_proj = Linear(d_model, d_model)

```

The Forward Pass (Step by Step)

Input: $x \in \mathbb{R}^{(B \times L \times D)}$ where B =batch, L =length, D = d_model

Step 1: Expand and split

```

xz = in_proj(x) # (B, L, 2D)
x, z = split(xz) # each (B, L, D)

```

Step 2: Local 1D convolution

```
x = conv1d(x) # (B, L, D)
x = SiLU(x)
```

Why convolution? Captures local dependencies (like n-grams in text). This is cheap (kernel size ~ 4) and helps with short-range patterns.

Step 3: Compute SSM parameters

```
x_db = x_proj(x) # (B, L, d_state*2 + d_conv)
Δ, B, C = split(x_db, [d_conv, d_state, d_state])

Δ = softplus(dt_proj(Δ)) # (B, L, D), ensure Δ > 0
```

Step 4: Selective SSM (the core!)

```
A = -exp(A_log) # (D, N), ensure eigenvalues negative
y = selective_scan(x, Δ, A, B, C) # (B, L, D)
y = y + D * x # skip connection
```

Step 5: Gate and project

```
y = y * SiLU(z) # gating with z pathway
output = out_proj(y) # (B, L, D)
```

5.2 The Selective Scan Algorithm

This is the heart of Mamba. Here's what happens inside:

```
def selective_scan(x, Δ, A, B, C):
    # x: (B, L, D)
    # Δ: (B, L, D) - per-channel step size
    # A: (D, N) - fixed HiPPO matrix
```

```

# B: (B, L, N) - input projection
# C: (B, L, N) - output projection

# Discretize per position and channel
ΔA = exp(Δ.unsqueeze(-1) * A) # (B, L, D, N)
ΔB = Δ.unsqueeze(-1) * B.unsqueeze(2) # (B, L, D, N)

# Parallel scan (or recurrent during inference)
h = parallel_scan(ΔA, ΔB, x) # (B, L, D, N)

# Output
y = (h * C.unsqueeze(2)).sum(-1) # (B, L, D)
return y

```

Parallel Scan Intuition

Think of the recurrence as a binary tree computation:

Level 0:	h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8
	↓	↓	↓	↓	↓	↓	↓	↓
Level 1:	h_{1-2}		h_{3-4}		h_{5-6}		h_{7-8}	
	↓		↓		↓		↓	
Level 2:	h_{1-4}				h_{5-8}			
	↓				↓			
Level 3:	h_{1-8}							

At each level, we combine adjacent segments using the associative property:

$$h_{\{a:b\}} = \bar{A}_{\{a:b\}} * h_{\{0\}} + (\text{combined B terms})$$

This takes $O(\log L)$ levels with more work per level, but can be parallelized on GPU.

5.3 Key Design Choices Explained

Choice 1: Why d_state (N) is small (16-64)?

- Transformers have hidden dimension ~1000+
- Mamba's SSM state is only ~16-64 dimensions

Reason: The state h is a **compressed representation** optimized via HiPPO. It doesn't need to be large because it's structured. The "expressiveness" comes from:

1. Input-dependent dynamics (selectivity)
2. Per-channel processing (D independent SSMs)
3. Multiple layers stacking

Choice 2: Why per-channel SSMs (D independent SSMs)?

Each of the D channels runs its own independent SSM with shared A but different B, C, Δ . This is like having D parallel "memory banks" that can specialize:

- Some channels might learn to track syntax
- Some might track entities
- Some might track long-range coreferences

This is parameter-efficient parallelism.

Choice 3: Why the skip connection D?

$$y = \text{SSM}(x) + D * x$$

The D parameter is a learned scalar that adds a direct path from input to output. This helps with:

- Gradient flow (like ResNet)
- Preserving high-frequency details that SSM might smooth out

- Balancing between sequential processing and direct information

Choice 4: Why gating with z?

$$y = \text{SSM_output} * \text{SiLU}(z)$$

This is borrowed from gated MLPs (SwiGLU, GeGLU). The z pathway:

- Sees the same input as x
- Provides input-dependent gating
- Adds capacity (doubles parameters in expansion)
- Works well empirically

5.4 Complete Mamba Model Architecture

A full Mamba language model stacks these blocks:

```
Input Embeddings (vocab_size → d_model)
    ↓
[Mamba Block 1]
    ↓
[RMSNorm]
    ↓
[Mamba Block 2]
    ↓
[RMSNorm]
    ↓
... (N layers)
    ↓
[RMSNorm]
    ↓
Output Head (d_model → vocab_size)
```

No positional encodings needed! The SSM's temporal dynamics naturally handle position.

Typical configuration (Mamba-2.8B):

- $d_{\text{model}} = 2560$
- $d_{\text{state}} = 16$
- $d_{\text{conv}} = 4$
- $n_{\text{layers}} = 64$

5.5 Training Considerations

Memory Efficiency

During training, Mamba uses **recomputation** for the selective scan:

- Don't store all intermediate h states (would be huge)
- Recompute them during backward pass
- Trade computation for memory

Gradient Flow

Gradients flow through:

1. The SSM path (through h states via backpropagation through time)
2. The gating path (direct)
3. The skip connection D (direct)

Multiple paths help with gradient flow, similar to ResNet.

Initialization

- A: HiPPO initialization (crucial!)
- B, C, Δ projections: careful initialization to start near identity-like behavior

- D: initialized to ~ 1

5.6 Empirical Performance

Mamba achieves competitive performance with Transformers on:

- **Language modeling:** Similar perplexity on large-scale datasets
- **Long context:** Better on tasks requiring 10K+ context (vs. Transformers with $O(L^2)$ attention)
- **Inference speed:** 5-10 $\times$ faster than Transformers for long sequences
- **DNA modeling:** Excels at long genomic sequences (1M+ tokens)

Where Mamba Struggles

- **Copying tasks:** Attention's explicit all-to-all routing is better for exact recall
- **In-context learning:** Transformers still slightly better at few-shot prompting
- **Very large scale:** Transformers still dominate at 100B+ parameter scale (more research needed)

Comprehensive Final Quiz

Let's test everything you've learned:

Q1 (Architecture): Mamba uses both a 1D convolution (kernel size ~ 4) AND a selective SSM. What complementary roles do these two components play?

Q2 (Mathematics): In the selective scan, we compute $\bar{A}_k = \exp(\Delta_k \cdot A)$ where A is fixed (HiPPO) but Δ_k varies. Why is it important that A has negative eigenvalues?

Q3 (Complexity): During training, Mamba uses parallel scan which has more operations than S4's FFT convolution. Yet Mamba is marketed as "efficient." Where

does the efficiency come from?

Q4 (Design): Compare the information flow in:

- Transformer: Q, K, V projections → attention → output
- Mamba: $x \rightarrow \{\Delta, B, C\}$ projections → selective SSM → gating

What's fundamentally different about how they route information?

Q5 (Conceptual): Why is Mamba called "selective"? What exactly is being selected, and how does this relate to attention mechanisms?

Take your time with these - they require integrating everything we've covered!

⊗ A1: the 1D convolution acts as a local, static feature extractor, while the selective SSM acts as a global, dynamic state manager. A2: This is essential for guaranteeing the stability of the recurrent system. If the eigenvalues of A weren't negative, the hidden state h would explode to infinity. A4: For Transformer, every token broadcasts a query and searches the entire sequence, pulling information from any other token based on a learned similarity score. Mamba uses state-based compression. Every token updates a single, compressed hidden state, and the routing is controlled by that token's decision to save, forget, or pass through information. A5: Mamba is called "selective" because its internal evolution depends on the input content. Each token selects what dynamics to apply and which parts of past memory to retrieve. So it's like doing implicit selection by modifying the recurrence so that the dynamics themselves depend on content, causing the model to selectively propagate or forget information over time. While Transformer can be viewed as performing explicit selection by comparing all tokens to decide what to attend to. Mamba achieves content-dependent information flow (like attention) but with the efficiency of no quadratic cost as that of Transformer.

[These questions really help me gain a deeper understanding of the design of Mamba architecture](#)

Thought: Lauded exceptional student comprehension, identified missing response, prepared comprehensive feedback.

Exceptional answers! Your responses show you've integrated the entire conceptual framework. Let me give specific feedback:

A1: Perfect. "Local/static vs global/dynamic" captures the complementary design beautifully. The conv handles short-range patterns (like character/word n-grams), while SSM handles arbitrarily long dependencies.

A2: Exactly right. You've connected the math (negative eigenvalues) → continuous stability → discrete stability after $\exp(\Delta A)$. This is the foundation that makes the whole system work.

A4: Brilliant framing! "Broadcast-and-search vs state-based compression" is an elegant way to distinguish the architectures. Transformers create temporary routing (attention weights), while Mamba creates persistent compression (state h).

A5: Outstanding! You distinguished **explicit selection** (attention computes who-attends-to-whom) from **implicit selection** (Mamba modulates dynamics to control information flow). This is a sophisticated insight that many papers don't articulate clearly.

However, you skipped Q3! Let me address it since it's important:

Q3 Answer: Where Does Mamba's Efficiency Come From?

Despite parallel scan having more operations than FFT convolution, Mamba achieves efficiency through:

1. Hardware-Aware Implementation (Kernel Fusion)

- Mamba fuses the entire selective scan into a single GPU kernel
- No intermediate results written to HBM (high-bandwidth memory)
- Everything stays in fast SRAM (on-chip cache)
- This is the **key difference**: S4's FFT requires multiple kernel launches and HBM reads/writes

2. IO Awareness

Modern GPU computation is often **memory-bound**, not compute-bound. Operations per second matters less than memory bandwidth usage.

- FFT: Many memory transfers, even if fewer FLOPs
- Parallel scan: More FLOPs but better memory locality when fused

3. No Materialization of K

S4 must materialize the convolution kernel $\tilde{K}$ (length L) in memory. For $L=100K$, this is expensive. Mamba never materializes the full kernel.

4. Inference Advantage

During inference (where it matters most for deployment):

- Mamba: $O(1)$ per token, constant memory
- Transformer: $O(L)$ per token, growing KV cache
- This compounds over long generation

The lesson: Algorithmic complexity (big-O) doesn't tell the full story. Hardware characteristics (memory hierarchy, kernel fusion) matter enormously for deep learning.

Part 6: Putting It All Together - The Big Picture

You've now learned the complete journey:

The Evolution

RNNs (1980s-2010s)

↓

Problem: Sequential, vanishing gradients

↓

Transformers (2017)

↓

Problem: $O(L^2)$ complexity

↓

S4 (2022) - Structured State Spaces

↓

Problem: Input-invariant (not selective)

↓

Mamba (2023) - Selective State Spaces

What Mamba Achieves

Desirable Property	RNN	Transformer	S4	Mamba
Parallel training	✗	✓	✓	✓
Linear complexity	✓	✗	✓	✓
Constant inference	✓	✗	✓	✓
Long-range memory	✗	✓	✓	✓
Content-based routing	⚠	✓	✗	✓

Mamba gets all five checkmarks!

The Core Innovations Recap

1. **HiPPO (from S4):** Principled initialization for long-range dependencies
2. **Selectivity (Mamba):** Input-dependent dynamics $\{\Delta, B, C\}$
3. **Hardware-aware scan (Mamba):** Efficient implementation via kernel fusion
4. **Hybrid architecture (Mamba):** Conv + SSM + gating for expressiveness

When to Use Mamba vs Transformers

Use Mamba when:

- Very long sequences (100K+ tokens)



- Inference latency matters (chatbots, real-time systems)
- Memory constraints (edge deployment)
- Sequential data with clear temporal structure (audio, DNA, time series)

Use Transformers when:

- Need maximum performance (at scale, Transformers still edge out)
- Heavy in-context learning (few-shot prompting)
- Rich ecosystem matters (more tools, libraries, pretrained models)
- Tasks requiring explicit all-to-all comparisons (retrieval, matching)

The future: Hybrid models combining both! Some recent work explores using Mamba for the "backbone" and attention for specific layers.

Final Mastery Check

One last conceptual question to test complete understanding:

Synthesis Question:

Imagine you're designing a new model for processing DNA sequences (millions of base pairs). You need to:

- Detect long-range regulatory interactions (10K+ base pairs apart)
- Run efficiently on limited GPU memory
- Handle sequences up to 1M tokens

Would you choose Mamba or Transformer? Justify your answer by discussing:

1. How each model would handle the long-range dependencies
2. The memory/compute tradeoffs at 1M sequence length
3. What properties of DNA sequences might favor one architecture

This requires you to apply everything you've learned to a real scenario!

Further Learning Resources

Since you've mastered the concepts, here are next steps:

Papers to Read (in order):

1. **"Mamba: Linear-Time Sequence Modeling with Selective State Spaces"** (Gu & Dao, 2023) - The original paper
2. **"Hungry Hungry Hippos: Towards Language Modeling with State Space Models"** (Dao et al, 2022) - Predecessor work
3. **"Efficiently Modeling Long Sequences with Structured State Spaces"** (Gu et al, 2022) - The S4 paper
4. **"HiPPO: Recurrent Memory with Optimal Polynomial Projections"** (Gu et al, 2020) - Mathematical foundations

Implementation:

- Study the official Mamba implementation on GitHub (state-spaces/mamba)
- The `selective_scan_cuda` kernel is particularly instructive
- Try implementing a simple version yourself in PyTorch (without the CUDA optimizations first)

Related Work:

- **RWKV**: Another linear attention alternative
- **RetNet**: Microsoft's retentive networks
- **Hyena**: Combines convolutions with attention
- **Mamba-2**: Recent improvements to the original Mamba

You're now equipped to understand all of these!

Would you like me to answer the final synthesis question about DNA sequences, or would you like to try it yourself first? Also, are there any specific aspects you'd like to dive deeper into (implementation details, mathematical proofs, comparison with other architectures, etc.)?